

1. Prompt for Understanding The Project Structure

I'm working on a Python/Pygame project called "Maze Chase". Here is my GitHub repository: [YVLNair/11_maze_chase](https://github.com/YVLNair/11_maze_chase)

Please inspect the project structure and explain how the player, enemy, maze, GameEngine, and BFS pathfinding currently work. Don't make changes yet.

- Inspected the repository structure: `main.py`, `game/game_engine.py`, `game/entities.py`, and `game/maze.py`.
- **Player:** Handles keyboard input and player movement through the maze.
- **Enemy:** Tracks its maze-cell position and periodically chases the player using BFS.
- **Maze:** Generates the maze and stores wall information for each cell.
- **GameEngine:** Runs the main game loop and coordinates input, updates, rendering, collisions, win/game-over states, and restarting.
- **BFS:** Searches the maze's open cells to find a path from the enemy to the player and determines the enemy's next movement direction. github.com
- No changes were made.



2. Prompt for implementing Task 1

Implement Task 1: Multiple Enemies. Add two more enemies starting from different maze corners. Each enemy should independently use BFS to chase the player. Use a `self.enemies` list and update all enemies through the game engine. Keep the existing architecture.

Make the following changes in `game/game_engine.py`.

1. Replace the single enemy in `reset()`

Find:

```
</> Python
self.enemy = Enemy(ROWS - 1, COLS - 1)
```

Replace it with:

```
</> Python
self.enemies = [
    Enemy(0, COLS - 1),      # Top-right
    Enemy(ROWS - 1, 0),     # Bottom-left
    Enemy(ROWS - 1, COLS - 1) # Bottom-right
]
```

This gives you three independent enemies starting from different corners.

2. Update all enemies in `update()`

Find:

```
</> Python
self.enemy.update(self.walls, self.player, ROWS, COLS)
```

Replace it with:

```
</> Python
for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)
```

+ Ask anything

Think

🔊

🔊

Replace it with:

Python



Run

```
for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)
```

Each `Enemy` object independently runs its existing BFS logic.

3. Update collision detection

Find the existing single-enemy collision check:

Python



Run

```
if self.player.rect.colliderect(self.enemy.rect):
    self.caught = True
```

Replace it with:

Python



Run

```
for enemy in self.enemies:
    if self.player.rect.colliderect(enemy.rect):
        self.caught = True
        break
```

The player is caught if they collide with **any** enemy.

4. Update drawing

Find:

Python



Run

```
self.enemy.draw(self.screen)
```

Replace it with:

Python



Run

```
for enemy in self.enemies:
    enemy.draw(self.screen)
```



5. Remove remaining `self.enemy` references

+ Ask anything

Think



Python



Run

5. Remove remaining `self.enemy` references

After making the changes, search `game_engine.py` for:

</> Python



Run

```
self.enemy
```

There should be no remaining references. All enemy operations should now go through:

</> Python



Run

```
self.enemies
```

You don't need to modify the `Enemy` class or BFS implementation. Each object in `self.enemies` already maintains its own position and movement state, so the existing BFS behavior works independently for all three enemies.



ChatGPT can make mistakes. Check important info.

+ Ask anything

Think



3. Prompt for implementing Task 2

Implement Task 2: Speed Up Over Time. Use `pygame.time.get_ticks()` to track elapsed time. Every 15 seconds, decrease each enemy's `move_interval` by 2, with a minimum of 5. Also display the current speed tier in the HUD.

Make the following changes in `game/game_engine.py`.

1. Initialize the speed timer in `reset()`

Inside `reset()`, after creating the enemies, add:

```
</> Python
self.start_time = pygame.time.get_ticks()
self.speed_tier = 0
```

So the relevant section becomes:

```
</> Python
self.enemies = [
    Enemy(0, COLS - 1),
    Enemy(ROWS - 1, 0),
    Enemy(ROWS - 1, COLS - 1)
]

self.start_time = pygame.time.get_ticks()
self.speed_tier = 0
```

This makes the 15-second timer restart whenever the game is restarted.

2. Add the speed-up logic to `update()`

At the beginning of `update()`, after your game-over check:

```
</> Python
if self.caught or self.won:
    return
```

add:

+ Ask anything

Think



```
elapsed = pygame.time.get_ticks() - self.start_time
```

2. Add the speed-up logic to `update()`

At the beginning of `update()`, after your game-over check:

`</>` Python



Run

```
if self.caught or self.won:
    return
```

add:

`</>` Python



Run

```
elapsed = pygame.time.get_ticks() - self.start_time
current_tier = elapsed // 15000

if current_tier > self.speed_tier:
    for enemy in self.enemies:
        enemy.move_interval = max(5, enemy.move_interval - 2)

    self.speed_tier = current_tier
```

So it should look like:

`</>` Python



Run

```
def update(self):
    if self.caught or self.won:
        return

    elapsed = pygame.time.get_ticks() - self.start_time
    current_tier = elapsed // 15000

    if current_tier > self.speed_tier:
        for enemy in self.enemies:
            enemy.move_interval = max(5, enemy.move_interval - 2)

        self.speed_tier = current_tier

    # existing player/enemy update code continues here
```

The progression will be:

```
0 sec → Tier 0 → interval 20
15 sec → Tier 1 → interval 18
30 sec → Tier 2 → interval 16
```



+ Ask anything

Think



Once the interval reaches 5, it won't decrease any further

Once the interval reaches `5`, it won't decrease any further.

3. Add the speed tier to the HUD

Make sure your `__init__()` has a HUD font:

```
Python
self.hud_font = pygame.font.SysFont("monospace", 16)
```

Then in `draw()`, replace your existing HUD text with:

```
Python
info = self.hud_font.render(
    f"Speed Tier: {self.speed_tier}    R=Restart",
    True,
    (200, 200, 200)
)

self.screen.blit(
    info,
    (8, ROWS * CELL + 17)
)
```

If you already have `self.hud_font`, don't create another local `hud_font` inside `draw()`.

Result

Task 2 will now:

- Track elapsed time using `pygame.time.get_ticks()`.
- Increase the speed tier every 15 seconds.
- Decrease **every enemy's** `move_interval` by 2.
- Never allow the interval to go below 5.
- Reset the timer and tier when `R` restarts the game.
- Display the current speed tier in the HUD.

📄 📌 📤 🔄 ...

ChatGPT can make mistakes. Check important info.

+ Ask anything

🧠 Think

🔊

🔊

4. Prompt for implementing Task 3

Implement Task 3: Power Pellet. Add a yellow power pellet in a valid maze location. When the player collects it, freeze the enemies for 5 seconds. While frozen, enemies should not move, and a frozen indicator should be visible.

Make these changes while keeping Tasks 1 and 2 intact.

1. `game/game_engine.py` — add pellet state in `reset()`

Inside `reset()`, after creating the enemies, add:

```
Python

pellet_r = ROWS // 2
pellet_c = COLS // 2 - 2

self.pellet_rect = pygame.Rect(
    pellet_c * CELL + CELL // 2 - 7,
    pellet_r * CELL + CELL // 2 - 7,
    14,
    14
)

self.pellet_collected = False
self.freeze_end_time = 0
```

Your enemy setup should remain:

```
Python

self.enemies = [
    Enemy(0, COLS - 1),
    Enemy(ROWS - 1, 0),
    Enemy(ROWS - 1, COLS - 1)
]
```

Keep your existing Task 2 timer code:

```
Python

self.start_time = pygame.time.get_ticks()
self.speed_tier = 0
```

+ Ask anything

Think



2. `game/game_engine.py` — detect pellet collection

2. `game/game_engine.py` — detect pellet collection

In `update()`, after your player movement code, add:

```
Python
if (
    not self.pellet_collected
    and self.player.rect.colliderect(self.pellet_rect)
):
    self.pellet_collected = True
    self.freeze_end_time = pygame.time.get_ticks() + 5000

    for enemy in self.enemies:
        enemy.frozen = True
```

3. `game/game_engine.py` — unfreeze after 5 seconds

Also in `update()`, add:

```
Python
if (
    self.pellet_collected
    and self.freeze_end_time > 0
    and pygame.time.get_ticks() >= self.freeze_end_time
):
    for enemy in self.enemies:
        enemy.frozen = False

    self.freeze_end_time = 0
```

This keeps the enemies frozen for exactly 5 seconds.

4. Prevent frozen enemies from moving

Find your existing Task 1 enemy update:

```
Python
for enemy in self.enemies:
    enemy.update(self.walls, self.player, ROWS, COLS)
```

+ Ask anything

Think



Replace it with:

```
</> Python
for enemy in self.enemies:
    if not enemy.frozen:
        enemy.update(self.walls, self.player, ROWS, COLS)
```

The enemies still exist and are drawn, but their movement/BFS update is skipped while frozen.

5. `game/entities.py` — add frozen state

Inside `Enemy.__init__()`, add:

```
</> Python
self.frozen = False
```

For example:

```
</> Python
def __init__(self, r, c):
    self.r, self.c = r, c
    self.timer = 0
    self.move_interval = 20
    self.frozen = False
    # existing code...
```

6. `game/entities.py` — show frozen indicator

In `Enemy.draw()`, change the color used to draw the enemy.

For example:

```
</> Python
def draw(self, screen):
    draw_color = (80, 170, 255) if self.frozen else self.color

    pygame.draw.rect(
        screen,
```

+ Ask anything

Think



6. `game/entities.py` — show frozen indicator

In `Enemy.draw()`, change the color used to draw the enemy.

For example:

```
Python Copy Run

def draw(self, screen):
    draw_color = (80, 170, 255) if self.frozen else self.color

    pygame.draw.rect(
        screen,
        draw_color,
        self.rect,
        border_radius=5
    )

    # keep your existing eyes/details here
```

Normal enemies keep their existing color, while frozen enemies become visibly blue.

7. `game/game_engine.py` — draw the pellet

In `draw()`, before drawing the player/enemies, add:

```
Python Copy Run

if not self.pellet_collected:
    pygame.draw.circle(
        self.screen,
        (255, 220, 0),
        self.pellet_rect.center,
        7
    )
```

So the pellet disappears after being collected.

Final behavior

```
Game starts
↓
Yellow pellet appears
↓
```



+ Ask anything

Think



Final behavior

```
Game starts
↓
Yellow pellet appears
↓
Player touches pellet
↓
Pellet disappears
↓
All 3 enemies become frozen
↓
Enemies show frozen indicator
↓
5 seconds pass
↓
Enemies become normal and resume BFS movement
```



Your existing **multiple enemies** and **speed-up system** remain unchanged.



ChatGPT can make mistakes. Check important info.

+ | Ask anything

Think



5. Prompt for implementing Task 4

Implement Task 4: Score by Distance. Increase the player's survival score by 1 every frame while alive. Display `Survived: Xs` in the HUD using `score // 60`, and show the final score on the game-over screen.

Make these changes in `game/game_engine.py` while keeping Tasks 1–3 unchanged.

1. Initialize the score in `reset()`

Add:

```
</> Python
self.score = 0
```

For example, near your existing Task 2 reset values:

```
</> Python
self.start_time = pygame.time.get_ticks()
self.speed_tier = 0
self.score = 0
```

This resets the survival score whenever the game restarts.

2. Increase the score every frame while alive

At the end of `update()`, after your existing movement, pellet, enemy, collision, and win logic, add:

```
</> Python
if not self.caught and not self.won:
    self.score += 1
```

Your existing beginning of `update()` should still have:

```
</> Python
if self.caught or self.won:
    return
```

This ensures the score stops increasing once the player has been caught or escaped.

+ Ask anything

Think



3. Change the HUD

Replace your current HUD text with:

```
</> Python   Run

info = self.hud_font.render(
    f"Survived: {self.score // 60}s   Speed Tier: {self.speed_tier}   R=Restart",
    True,
    (200, 200, 200)
)

self.screen.blit(
    info,
    (8, ROWS * CELL + 17)
)
```

If the text doesn't fit, reduce the HUD font size:

```
</> Python   Run

self.hud_font = pygame.font.SysFont("monospace", 13)
```

4. Show the final survival time on the game-over screen

Update your `_overlay()` function so it includes the score:

```
</> Python   Run

def _overlay(self, text, color):
    surf = pygame.Surface((WIDTH, ROWS * CELL), pygame.SRCALPHA)
    surf.fill((0, 0, 0, 140))
    self.screen.blit(surf, (0, 0))

    msg = self.big_font.render(text, True, color)

    score_text = self.font.render(
        f"Survived: {self.score // 60}s",
        True,
        (200, 200, 200)
    )

    restart_text = self.font.render(
        "Press R to Restart",
        True,
```



+ Ask anything

 Think



```
self.screen.blit(
```

```

restart_text = self.font.render(
    "Press R to Restart",
    True,
    (200, 200, 200)
)

self.screen.blit(
    msg,
    (
        WIDTH // 2 - msg.get_width() // 2,
        ROWS * CELL // 2 - 30
    )
)

self.screen.blit(
    score_text,
    (
        WIDTH // 2 - score_text.get_width() // 2,
        ROWS * CELL // 2 + 15
    )
)

self.screen.blit(
    restart_text,
    (
        WIDTH // 2 - restart_text.get_width() // 2,
        ROWS * CELL // 2 + 42
    )
)

```

Result

- Score starts at `0` on every restart.
- Score increases by `1` every game frame while alive.
- HUD displays `Survived: Xs` using `score // 60`.
- Score stops increasing after **CAUGHT** or **ESCAPED**.
- Final survival time appears on the game-over/escape overlay.
- Tasks 1–3 remain intact.

📄 📌 📤 🔄 ...

ChatGPT can make mistakes. Check important info.

+ Ask anything

🤖 Think

🎤

🔊